PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Phạm Anh Tài MSSV: N24DCDT081 Lớp: D24CQDT01-N

GitHub: https://github.com/tai2006hvn-spec/thuchanhbuoi5.git Commit SHA: a8c9bd6

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

- Mục tiêu: Điều phối giao các đơn hàng độc lập từ kho xuất phát (điểm 1) đến điểm giao rồi quay về kho sao cho tổng lợi ích đạt cực đại trong ngân sách thời gian cho phép.
- Đầu vào: don_hang.csv (danh mục đơn), ban_do.csv (đồ thị không trọng số các điểm 1..6 và các cạnh), ngân sách thời gian B.
- Đầu ra: Khoảng cách ngắn nhất và chi phí thời gian tau = 2h + service cho từng đơn, danh sách đơn tối ưu được chọn, trạng thái điều phối qua hàng đợi vòng FIFO và ngăn xếp LIFO, tệp ket_qua.csv.
- Sơ đồ truyền dữ liệu: Đọc dữ liệu CSV -> BFS tìm đường ngắn nhất & tính chi phí tau -> Quy hoạch động Ba lô 0/1 chọn đơn tối ưu -> Sắp xếp trộn Merge Sort theo tỉ số ưu tiên -> Nạp hàng đợi FIFO -> Giao đơn / Hoàn tác (Stack) & Tra cứu (Binary Search / BST) -> Xuất ket_qua.csv.

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 -> 2	1	3
2	1 -> 2 -> 4	2	5
3	1 -> 2 -> 4 -> 6	3	7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Chia đôi dãy đệ quy đến khi độ dài <= 1, sau đó trộn hai dãy con đã có thứ tự. Ưu tiên nhánh trái khi hai phần tử hòa nhau để đảm bảo tính ổn định (stable).

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=sort -> ĐẠT 8/8 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(n log n); bộ nhớ phụ O(n).

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng/điều kiện áp dụng: Áp dụng trên dãy Order đã sắp xếp tăng dần theo ID. So sánh ID với phần tử giữa mid = lo + (hi - lo) div 2 để thu hẹp một nửa khoảng tìm kiếm. Trả về Order nếu thấy, nothing nếu không có.

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=search -> ĐẠT 5/5 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(log n); bộ nhớ phụ O(1).

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng/điều kiện áp dụng: Duyệt BFS đồ thị không trọng số bằng hàng đợi vòng CircularQueue. Đánh dấu distance và parent ngay khi phát hiện đỉnh để tránh lặp. Dựng đường đi bằng cách lần ngược từ đích về nguồn qua parent.

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs -> ĐẠT 7/7 phép kiểm tra (khoảng cách [0,1,1,2,2,3]).

Chi phí thời gian/bộ nhớ: Thời gian O(V + E); bộ nhớ phụ O(V).

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng/điều kiện áp dụng: Quy hoạch động Ba lô 0/1 bảng kích thước (n+1) x (budget+1). Công thức: table[i+1, b+1] = max(table[i, b+1], benefit + table[i, b - cost + 1]). Khi hòa lợi ích, ưu tiên không lấy đơn đang xét. Truy vết từ ô (n, budget) về (0,0).

Lời gọi và kết quả kiểm thử: julia TH_2026_10_09_Kiem_thu.jl --phase=dp -> ĐẠT 12/12 phép kiểm tra.

Chi phí thời gian/bộ nhớ: Thời gian O(n * B); bộ nhớ phụ O(n * B).

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

- Ngân sách B = 8; Tập đơn được chọn: [1, 2]; Tổng thời gian: 8; Tổng lợi ích: 14.
- Giải thích ô DP: Ô F(2,8) tương ứng table[3,9] trong Julia. Xét 2 đơn đầu với ngân sách 8: Không lấy đơn 2 thì nhận F(1,8) = 6; Có lấy đơn 2 thì nhận lợi ích 8 cộng với F(1, 8 - 5 = 3) = 6 (tại table[2,4]), tổng là 8 + 6 = 14. Chọn max(6, 14) = 14.

- Bước truy vết: Tại i=3, b=8 có table[4,9] == table[3,9] == 14 nên không lấy đơn 3. Tại i=2, b=8 có table[3,9] = 14 > table[2,9] = 6 nên chọn đơn 2, ngân sách còn 8 - 5 = 3. Tại i=1, b=3 chọn đơn 1.
- Đối chiếu vét cạn: Vét cạn 2^3 = 8 trường hợp cho kết quả tối ưu đúng bằng 14 tại tập [1, 2].

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1, 2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1, 2]	[]	0	0

6. Đầu ra và nhận xét

Đường dẫn CSV kết quả: data/MINI001/ket_qua.csv (và data/N24DCDT081/ket_qua.csv) Link notebook: TH_2026_10_09_Pluto.jl

Lỗi đã phát hiện và cách sửa: Trong Julia chỉ số mảng bắt đầu từ 1, do đó bảng DP cần tạo kích thước (n+1) x (budget+1) và truy xuất tại table[i+1, b+1] thay vì [i, b].

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào? Nếu bản đồ thay đổi, BFS sẽ tìm ra khoảng cách khác -> chi phí thời gian tau thay đổi -> đầu vào bài toán Ba lô thay đổi kéo theo tập nghiệm tối ưu thay đổi.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

- Lý do: Khóa sắp xếp của dãy ưu tiên là tỉ số (lợi ích / chi phí tau), hoàn toàn khác với thứ tự của trường Order.id. Nếu tìm kiếm nhị phân theo ID trên dãy này, tính chất tăng dần của ID bị mất, việc so sánh với phần tử giữa mid sẽ thu hẹp sai nửa, dẫn đến không tìm thấy đơn dù đơn đó có tồn tại.
- Tiền điều kiện: Dãy đầu vào bắt buộc phải được sắp xếp tăng dần theo trường khóa tìm kiếm (Order.id) và các ID phải là duy nhất.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

- Giải thích từng thành phần của tau: tau = 2*h + service = 2*2 + 1 = 5. Trong đó h = 2 là số cạnh ít nhất từ kho 1 tới điểm 4 (tuyến 1 -> 2 -> 4); 2*h = 4 là tổng thời gian di chuyển cả hai lượt đi và về; service = 1 là thời gian dừng phục vụ tại điểm giao.
- BFS có cho thời gian ngắn nhất khi cạnh khác nhau không: KHÔNG. BFS chỉ đúng với đồ thị không trọng số (mỗi cạnh tốn 1 đơn vị thời gian). Khi các cạnh có thời gian khác nhau, đường ít cạnh nhất chưa chắc có tổng thời gian nhỏ nhất; khi đó cần dùng thuật toán Dijkstra.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

- Vị trí trong table Julia: Ô table[2+1, 8+1] = table[3, 9].
- Hai phương án tạo ô F(2,8): Không lấy đơn 2 thì nhận F(1,8) = 6 (tại table[2,9]); Có lấy đơn 2 thì nhận lợi ích 8 cộng với F(1, 8 - 5 = 3) = 6 (tại table[2,4]), tổng là 8 + 6 = 14. Chọn max(6, 14) = 14.
- Vì sao nhánh lấy đơn phải dùng hàng trước: Đây là bài toán Ba lô 0/1 (mỗi đơn chỉ chọn tối đa 1 lần). Dùng hàng i-1 đảm bảo đơn i chưa từng được chọn ở các bước trước trong cùng mức ngân sách đó.

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

- Sự thay đổi trạng thái: Giao ID 1 làm lấy ID 1 ra khỏi FIFO, đẩy vào Stack, thời gian tăng 3, lợi ích tăng 6. Hoàn tác ngay rút ID 1 khỏi Stack, đẩy ngược lại đầu FIFO, trừ 3 thời gian và trừ 6 lợi ích. Trạng thái trở về ban đầu: Queue = [1, 2], Stack = [], Đã dùng = 0, Đã nhận = 0.
- Bất biến: Set(queue) hợp với Set(stack) luôn bằng đúng Set(selected_ids), giao nhau bằng rỗng, và tổng chi phí của các đơn trong stack luôn bằng đúng biến spent.

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

- Độ phức tạp: Sắp xếp trộn: thời gian O(n log n), bộ nhớ O(n). Tìm kiếm nhị phân: thời gian O(log n), bộ nhớ O(1). BFS cốt lõi: thời gian O(V + E), bộ nhớ O(V). DP bảng đầy đủ: thời gian O(n * B), bộ nhớ O(n * B).
- Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n): Vì cấu trúc cây BST phụ thuộc vào thứ tự chèn. Nếu chèn các khóa đã có thứ tự tăng dần hoặc

giảm dần, cây sẽ bị suy biến thành một danh sách liên kết có độ cao h = n, khi đó chi phí tìm kiếm xấu nhất là O(n).